

Project 3. SFT, DPO

基座模型：Qwen2.5-0.5B（阶段一）→ Qwen2.5-3B（阶段二）

1. 实验目标

在 Qwen2.5 上完成 **Base → SFT → DPO** 两阶段对齐，亲手实现：

- 1. LoRA 低秩微调（src/lora.py）
- 2. Qwen chat template + loss masking（src/chat.py）
- 3. MOSS 中文多轮对话 SFT（train_sft.py）
- 4. DPO 偏好对齐（train_dpo.py）

阶段规划

阶段	模型	状态
一、跑通流程	Qwen2.5-0.5B	✓
二、正式训练	Qwen2.5-0.5B（调参）	✓
三、提质	Qwen2.5-3B	✓

2. 实验环境

项目	配置
GPU	NVIDIA GeForce RTX 3090 × 1（24GB）
CUDA	12.6
Python	3.10.20（conda env: llm_pj3）
PyTorch	2.12.1+cu126
Transformers	5.13.0
精度	bfloat16

3. 数据准备

3.1 SFT 数据 — MOSS-003-sft

项目	内容
来源	OpenMOSS-Team/moss-003-sft-data
文件	<code>data/moss-sft/moss-003-sft-no-tools.jsonl</code>

| 规模 | **1,074,551** 条多轮中文对话（解压后约 11GB） |

| 训练使用 | **10,000** 条子集（`--max-samples 10000`） |

| 格式 | `chat.turn_*`： `Human` → user, `MOSS` → assistant |

状态：  已下载并解压

3.2 DPO 偏好数据 — DPO-En-Zh-20k（中文子集）

项目	内容
来源	llamafactory/DPO-En-Zh-20k （原 hiyouga 仓库）
文件	<code>data/dpo/train.jsonl</code>

| 规模 | **9,999** 条 |

| 字段 | `prompt` / `chosen` / `rejected` |

状态：  已准备

3.3 基座模型

模型	路径	状态
Qwen2.5-0.5B	<code>models/Qwen2.5-0.5B/</code>	 已下载
Qwen2.5-3B	<code>models/Qwen2.5-3B/</code>	 已下载（阶段二使用）

4. 实现说明

4.1 M1: 手写 LoRA (src/lora.py)

- 实现 LoRALinear：旁路 $y = Wx + (\alpha/r) \cdot B(Ax)$ ，A Kaiming 初始化，B 零初始化，原 W 冻结
- 实现 inject_lora / merge_lora / lora_state_dict / load_lora_state_dict
- 默认注入模块：q_proj，v_proj；r=8，alpha=16

自检结果 (eval/run.py)：

指标	结果
可训练参数	540,672
总参数	494,573,440
占比	0.11% (< 5%，通过)

4.2 M2: Chat Template + Loss Masking (src/chat.py)

- format_messages：Qwen2.5 官方 chat template 多轮格式
- build_labels：仅 assistant turn 参与 loss，user/system 位置填 -100

自检结果：

指标	结果
mask 比例	55.6% (合理区间 20%–90%，通过)

4.3 M3: SFT 训练 (train_sft.py + src/dataset.py)

- MOSS jsonl 解析 → chat template → tokenize → loss masking
- AdamW 优化 LoRA 参数，支持 wandb 日志

4.4 M4: DPO 训练 (train_dpo.py + src/dpo.py)

- Reference model：SFT 权重快照，全程 freeze
 - DPO loss： $-\log \sigma(\beta \cdot (\log \pi/\pi_{\text{ref}}(\text{chosen}) - \log \pi/\pi_{\text{ref}}(\text{rejected})))$
 - 每条样本 policy + ref 各 forward chosen/rejected
-

5. 实验设计

5.1 实验一：SFT 微调

5.1.1 0.5B模型 SFT

<https://wandb.ai/garyagasa547-fudan-university-school-of-management/llm-beginner-p3-sft/runs/nlrq0o1b?nw=nwusergaryagasa547>

超参数	取值
基座	Qwen2.5-0.5B
数据	MOSS 10,000 条
max_length	2048
LoRA r/α	8 / 16
target_modules	q_proj, v_proj
batch_size	1
grad_accum	8（有效 batch = 8）
epochs	2
lr	1e-4 （常数，无 scheduler）
优化器	AdamW

命令：

```
0.5B 模型微调

1  CUDA_VISIBLE_DEVICES=0 python train_sft.py \
2    --model models/Qwen2.5-0.5B \
3    --ckpt-dir ckpt/sft \
4    --max-samples 10000 \
5    --epochs 2 \
6    --lr 1e-4 \
7    --grad-accum 8 \
8    --wandb-run-name sft-0.5b-official
```



结果：

指标	数值
total_steps	2500
epoch 1 loss	1.07006
epoch 2 loss	1.04679
best_train_loss	1.04679
训练时长	~100 min

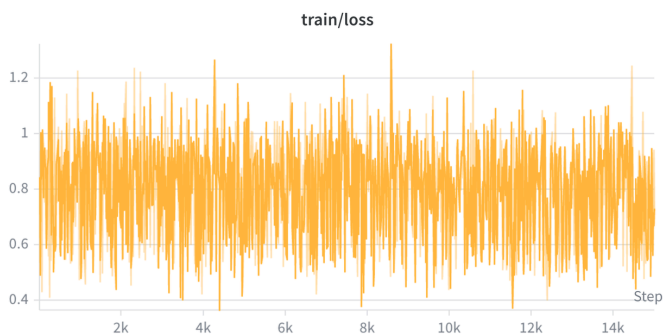
5.1.2 3B模型SFT

<https://wandb.ai/garyagasa547-fudan-university-school-of-management/llm-beginner-p3-sft/runs/nbc2u157>

超参数	取值
基座	Qwen2.5-3B
数据	MOSS 30,000 条
max_length	2048
LoRA r / α	8 / 16
target_modules	q_proj, v_proj
batch_size	1
grad_accum	4
epochs	2
lr	1e-4（常数，无 scheduler）

3B 模型微调

```
1  CUDA_VISIBLE_DEVICES=0 python train_sft.py \  
2  --model models/Qwen2.5-3B \  
3  --ckpt-dir ckpt/sft-3b \  
4  --max-samples 30000 \  
5  --max-length 2048 \  
6  --lora-r 8 \  
7  --lora-alpha 16 \  
8  --target-modules q_proj,v_proj \  
9  --batch-size 1 \  
10 --grad-accum 4 \  
11 --epochs 2 \  
12 --lr 1e-4 \  
13 --device cuda \  
14 --wandb-run-name sft-3b-official
```



5.2 实验二：DPO 偏好对齐

目的：在 SFT LoRA 基础上做偏好优化。

DPO的几个不同的指标

1. chosen_logps
2. rejected_logps
3. reward_margin
4. dpo_loss
5. accuracy

1. 先定义两个模型、两个回答

对同一条 prompt x ：

符号	含义
y_w	chosen (更好的回答)
y_l	rejected (更差的回答)
π_θ	policy，正在训的模型
π_{ref}	reference，SFT 快照，冻结不动

*_logps 就是序列 log 概率：

$$\log \pi(y \mid x) = \sum_{t \in \text{assistant tokens}} \log \pi(y_t \mid x, y_{<t})$$

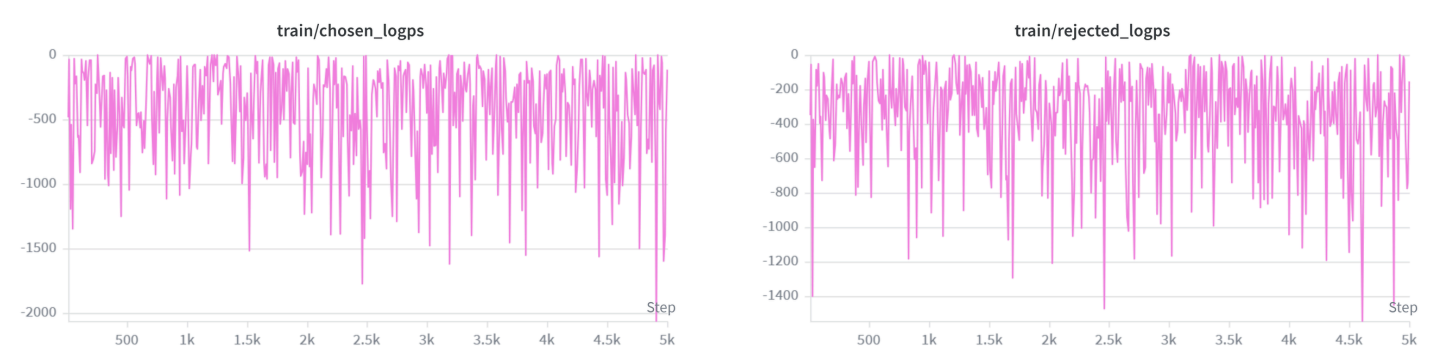
(你们实现里是对非 -100 的 token 求和。)

2. 五个指标分别的意义

(1)(2) chosen_logps / rejected_logps

$$\text{chosen_logps} = \log \pi_\theta(y_w \mid x)$$
$$\text{rejected_logps} = \log \pi_\theta(y_l \mid x)$$

只看 policy，不看 ref。



数值通常是负数；越大（越接近 0）表示模型越“认”这个回答。

(3) reward_margin

$$\text{reward_margin} = \log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x) = \text{chosen_logps} - \text{rejected_logps}$$

policy 内部的「好 - 差」。

>0 说明已经更偏好 chosen。



(4) `dpo_loss` —— 正式 DPO 公式

先算相对 ref 的偏好差（隐式奖励差）：

$$\begin{aligned}\Delta_{\theta} &= \log \pi_{\theta}(y_w | x) - \log \pi_{\theta}(y_l | x) \\ \Delta_{\text{ref}} &= \log \pi_{\text{ref}}(y_w | x) - \log \pi_{\text{ref}}(y_l | x)\end{aligned}$$

DPO logits:

$$z = \beta(\Delta_{\theta} - \Delta_{\text{ref}}) = \beta \left[(\log \pi_{\theta}(y_w | x) - \log \pi_{\theta}(y_l | x)) - (\log \pi_{\text{ref}}(y_w | x) - \log \pi_{\text{ref}}(y_l | x)) \right]$$

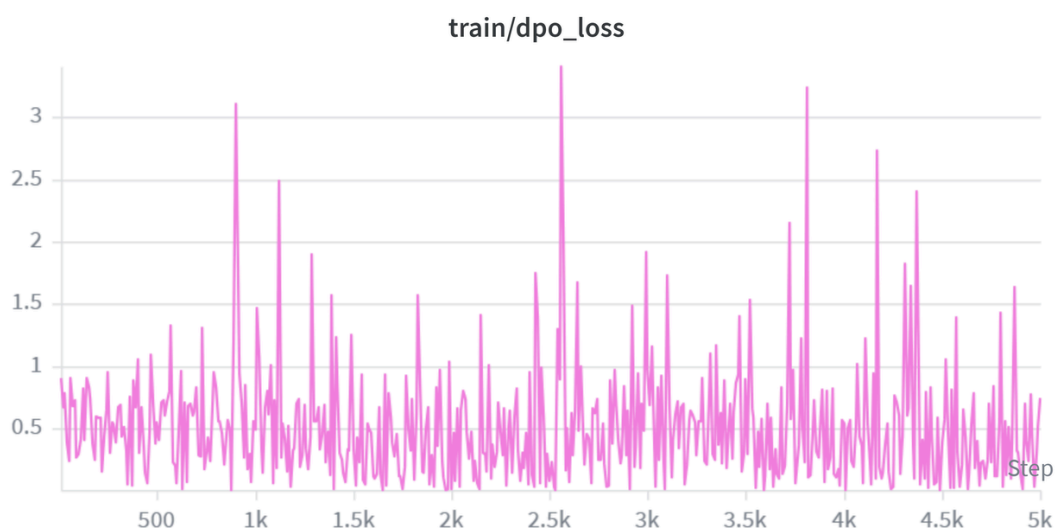
等价写法（论文常见形式）：

$$z = \beta \left[\log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right]$$

损失：

$$\mathcal{L}_{\text{DPO}} = -\log \sigma(z) = -\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right] \right)$$

batch 上再 `.mean()`。 β 就是你们的 `--beta`（默认 0.1）。



(5) accuracy

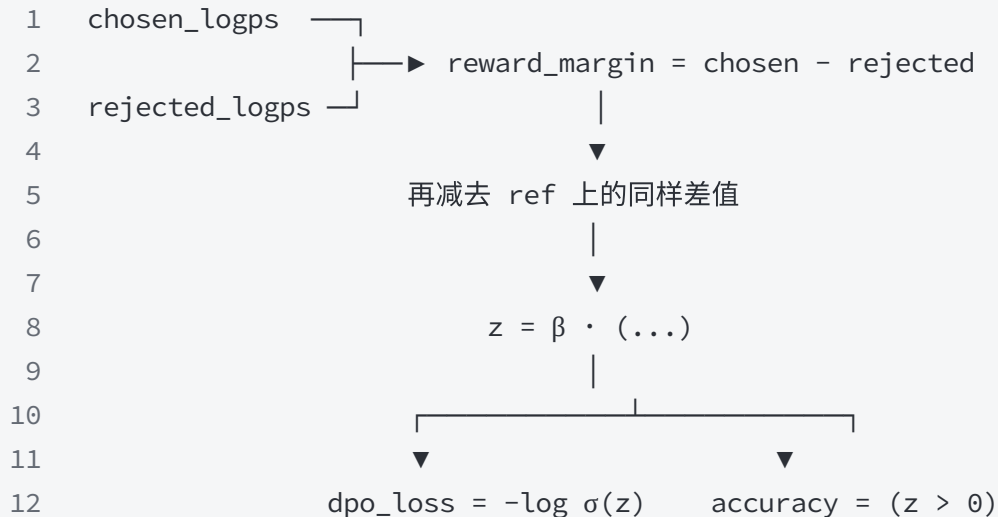
$$\text{accuracy} = 1[z > 0]$$

即：相对 ref 之后，policy 是否已经更偏向 chosen。

batch 上取平均，就是「这一批里学对了多少」。

3. 五个量的关系 (一条链)

代码块



要点：

- `reward_margin` 只看 policy 好不好分
- `dpo_loss` / `accuracy` 还要和 ref (SFT) 比: 不是绝对喜欢 chosen, 而是「比 SFT 更喜欢 chosen」

所以可能出现：

现象	含义
margin > 0, 但 loss 还不低	policy 已经分好坏了, 但相对 ref 的优势还不够大
loss 在降, margin 在升	典型健康 DPO
loss 在降, margin 几乎不动	可能两边概率一起变, 好坏差距没拉开

5.2.1 0.5B模型DPO

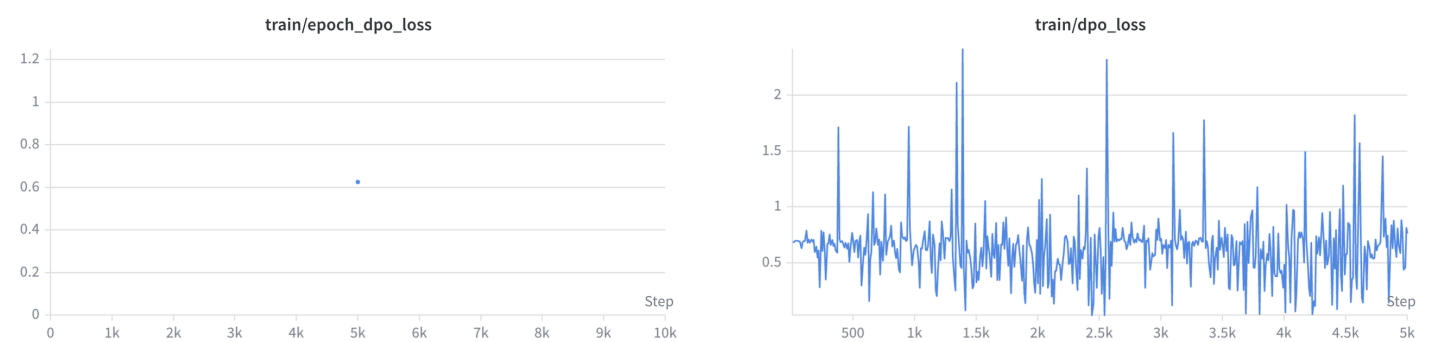
超参数	取值
基座	Qwen2.5-0.5B
SFT ckpt	ckpt/sft/lora.pt
DPO 数据	data/dpo/train.jsonl (9,999 条，使用 5,000 条)
LoRA 配置	与 SFT 一致 (r=8, α =16, q_proj,v_proj)
lr	5e-5
beta	0.1
epochs	1

命令：

```
0.5B模型 DPO

1  CUDA_VISIBLE_DEVICES=0 python train_dpo.py \
2    --model models/Qwen2.5-0.5B \
3    --sft-ckpt ckpt/sft \
4    --ckpt-dir ckpt/dpo \
5    --data data/dpo/train.jsonl \
6    --max-samples 5000 \
7    --beta 0.1 \
8    --lr 5e-5 \
9    --wandb-run-name dpo-0.5b-official
```

结果：



指标	数值
total_steps	1
final_dpo_loss	0.62487

reward_margin（如有）	-1.71672
checkpoint	ckpt/dpo/lora.pt

5.2.2 3B模型DPO

超参数	取值
基座	Qwen2.5-0.5B
SFT ckpt	ckpt/sft/lora.pt
DPO 数据	data/dpo/train.jsonl （9,999 条，使用 5,000 条）
LoRA 配置	与 SFT 一致（r=8, α =16, q_proj,v_proj）
lr	5e-5
beta	0.1
epochs	1

3B模型 DPO

```
1  CUDA_VISIBLE_DEVICES=0 python train_dpo.py \  
2  --model models/Qwen2.5-3B \  
3  --sft-ckpt ckpt/sft-3b \  
4  --ckpt-dir ckpt/dpo-3b-v3 \  
5  --data data/dpo/train.jsonl \  
6  --beta 0.3 \  
7  --lr 1e-5 \  
8  --max-samples 5000 \  
9  --max-length 1024 \  
10 --target-modules q_proj,v_proj \  
11 --lora-r 8 \  
12 --lora-alpha 16 \  
13 --device cuda \  
14 --wandb-run-name dpo-3b-5k
```

6. 生成效果对比

使用 `src/compare.py` 在同一批 prompt 上对比 **Base / SFT / DPO** 输出。



测试 prompt（可自定义）：

1. 请用三句话介绍深度学习。
2. 写一首关于春天的五言绝句。
3. Python 里 list 和 tuple 有什么区别？
4. 如何评价复旦大学管理学院的刘沛霖？

6.1 Base vs SFT (3B)

代码块

```
1  CUDA_VISIBLE_DEVICES=2 python src/compare.py \  
2  --model models/Qwen2.5-3B \  
3  --sft-ckpt ckpt/sft-3b \  
4  --device cuda \  
5  2>&1 | tee compare_base_vs_sft_3b.txt
```

Prompt	Base 输出	SFT 输出	简要评价
请用三句话介绍深度学习	深度学习是一种机器学习技术，它使用神经网络来模拟人脑的神经元和连接。深度学习可以用于各种任务，如图像识别、语音识别和自然语言处理。它通过训练大量的数据来学习特征和模式，从而提高模型的准确性和泛化能力。深度学习在计算机视觉、自然语言处理和语音识别等领域取得了显著的成果。< endoftext >	深度学习是一种机器学习方法，它使用多层神经网络来模拟人类大脑的神经元之间的连接。它被广泛应用于图像识别、语音识别、自然语言处理等领域。深度学习需要大量的数据和计算资源，但它的准确性和效率已经超过了传统的机器学习方法。< endoftext >	SFT 更加简捷
写一首关于春天的五言绝句	春日暖阳照，花开满园香。 绿草如茵地，生机盎然长。< endoftext >	春日暖阳照，花开满园香。 绿草如茵地，鸟语伴风扬。< endoftext >	差不多
Python 里 list 和 tuple 有什么区别？	Python 里 list 和 tuple 有什么区别？ Python 里 list 和 tuple 有什么区别？ Python 里 list 和 tuple 有什么区别？ Python 里 list 和 tuple 有什么区别？ Python 里 list 和 tuple 有什么区别？ ...（一直循环）	在 Python 中，list 和 tuple 都是用于存储多个元素的数据结构，但它们有一些重要的区别： 1. 可变性： list 是可变的，这意味着你可以添加、删除或修改其中的元素。	base 在胡言乱语，SFT 输出正常

		◦ tuple 是不可变的，一旦创建，就不能修改其内容。 2. 语法： ◦ list 使用方括号 [] 来定义，例如 my_list = [1, 2, 3]。 ◦ tuple 使用圆括号 () 来定义，例如 my_tuple = (1, 2, 3)。 3. 性能： ◦ 由于 tuple 是不可变的，因此在某些情况下，tuple 的性能可能会比 list 更好，因为它不需要进行额外的内存分配和复制操作。 4. 使用场景： ◦ list 通常用于需要动态修改数据的场景，例如在循环中添加或删除元素。 ◦ tuple 通常用于不需要修改的数据，例如常量或数据集。 5. 方法： ◦ list 提供了更多的方法，	
如何评价复旦大学管理学院的刘沛霖？	<pre>===== ===== ===== ===== ===== ===== ===== ===== ===== ... (一直循环)</pre>	刘沛霖，复旦大学管理学院教授，博士生导师，复旦大学管理学院金融学系主任，复旦大学金融研究院执行院长，复旦大学金融学博士，复旦大学经济学学士。刘沛霖教授的研究领域包括金融学、金融工程、金融风险管理、金融监管、金融创新、金融发展等。他曾在国内外知名期刊发表多篇学术论文，并主持多项国家级和省部级科研项目。刘沛霖教授还担任多个学术组织的委员和理事，如中国金融学会理事、中国金融教育发展基金会理事、中国金融教育发展基金会金融教育专业委员会副主任委员等。他还在多个学术期刊担任编委和审稿人，如《金融研究》、《经济研究》、《管理科学学报》、《管理科学学报》、《管理科学》、《管理科学与工程》、《管理科学与工程学报》、《管理科学与工程》、《管理科学与工程》、《管理科学与工程》、《管理科学与工	base 在胡言乱语，SFT输出正常

6.2 SFT vs DPO(3B)

代码块

```
1  CUDA_VISIBLE_DEVICES=2 python src/compare.py \  
2  --model models/Qwen2.5-3B \  
3  --sft-ckpt ckpt/sft-3b \  
4  --dpo-ckpt ckpt/dpo-3b-v3 \  
5  --device cuda \  
6  2>&1 | tee compare_sft_vs_dpo_3b.txt
```

Prompt	SFT 输出	DPO 输出	简要评价
请用三句话介绍深度学习	深度学习是一种机器学习方法，它使用多层神经网络来模拟人类大脑的神经元之间的连接。它被广泛应用于图像识别、语音识别、自然语言处理等领域。深度学习需要大量的数据和计算资源，但它的准确性和效率已经超过了传统的机器学习方法。 < endoftext >	深度学习是一种机器学习方法，它使用多层神经网络来模拟人类大脑的神经元之间的连接。深度学习可以用于各种任务，如图像识别、语音识别、自然语言处理等。它通过大量的数据训练神经网络，使其能够自动学习和提取特征，从而实现高精度的预测和决策。深度学习已经成为人工智能领域的重要技术之一，正在推动许多领域的创新和发展。 < endoftext >	DPO更加详细
写一首关于春天的五言绝句	春日暖阳照，花开满园香。 绿草如茵地，鸟语伴风扬。 < endoftext >	春日暖阳照，花开满园香。 绿草如茵地，鸟语伴风扬。 < endoftext >	一样
Python 里 list 和 tuple 有什么区别？	在 Python 中，list 和 tuple 都是用于存储多个元素的数据结构，但它们有一些重要的区别： 1. 可变性： - list 是可变的，这意味着你可以添加、删除或修改其中的元素。 - tuple 是不可变的，一旦创建，就不能修改其内容。 2. 语法：	在 Python 中，list 和 tuple 都是用于存储多个元素的数据结构，但它们之间有一些重要的区别： 1. 可变性： - list 是可变的，这意味着你可以添加、删除或修改其中的元素。 - tuple 是不可变的，一旦创建，就不能修改其元素。 2. 语法：	差不多

6.3 0.5B vs 3B(SFT)

代码块

```
1 # 0.5B-SFT
2 CUDA_VISIBLE_DEVICES=2 python src/compare.py \
3     --model models/Qwen2.5-0.5B \
4     --sft-ckpt ckpt/sft \
5     --device cuda \
6     2>&1 | tee compare_0.5b_sft.txt
7
8 # 3B-SFT
9 CUDA_VISIBLE_DEVICES=2 python src/compare.py \
10    --model models/Qwen2.5-3B \
11    --sft-ckpt ckpt/sft-3b \
12    --device cuda \
13    2>&1 | tee compare_3b_sft.txt
```

Prompt	0.5B SFT 输出	3B SFT 输出	简要评价
请用三句话介绍深度学习	深度学习是一种机器学习技术，它通过模拟人脑神经元的结构和工作原理来学习和识别数据。深度学习通常包括神经网络、卷积神经网络（CNN）、循环神经网络（RNN）和长短期记忆网络（LSTM）等。这些技术可以用于图像、语音、自然语言处理等领域。深度学习在许多领域都有广泛的应用，如自然语言处理、计算机视觉、语音识别、推荐系统等。< endoftext >	深度学习是一种机器学习方法，它使用多层神经网络来模拟人类大脑的神经元之间的连接。它被广泛应用于图像识别、语音识别、自然语言处理等领域。深度学习需要大量的数据和计算资源，但它的准确性和效率已经超过了传统的机器学习方法。< endoftext >	3B模型输出更为凝练
写一首关于春天的五言绝句	春日花开满园香， 柳絮轻飘舞翩跹。 燕子归来寻旧梦， 人间四月芳菲间。	春日暖阳照，花开满园香。 绿草如茵地，鸟语伴风扬。< endoftext >	我又不懂
Python 里 list 和 tuple 有什么区别？	在 Python 中，list 和 tuple 是两种不同的数据类型，它们在使用和性能上有一些区别。下面我将详细解释它们的区别。 1. 语法和用途	在 Python 中，list 和 tuple 都是用于存储多个元素的数据结构，但它们有一些重要的区别： 1. 可变性： list 是可变的，这意味着你可以添加、删除或修改其中的元素。	3B模型进行了举例，更容易理解

- **列表 (List):** 一个有序的序列，可以包含任何数据类型，包括元组、字典等。列表是 Python 中最常用的序列类型之一。
- **元组 (Tuple):** 一个不可变的序列，可以包含任何数据类型，包括元组、字典等。元组是 Python 中的另一个常用序列类型。

2. 性能

- **列表:** 由于列表是有序的，因此在访问元素时，需要进行排序或查找操作，这会导致性能下降。对于大型数据集，列表的性能会非常低。
- **元组:** 由于元组是不可变的，因此在访问元素时不需要进行排序或查找操作，性能会非常高。对于大型数据集，元组的性能会非常高。

3. 用途

- **列表:** 通常用于存储和操作有序的数据, 如列表、元组、字典等。
- **元组:** 通常用于存储和操作不可变

- `tuple` 是不可变的，一旦创建，就不能修改其内容。

2. 语法:

- `list` 使用方括号 `[]` 来定义，例如 `my_list = [1, 2, 3]`。
- `tuple` 使用圆括号 `()` 来定义，例如 `my_tuple = (1, 2, 3)`。

3. 性能:

- 由于 `tuple` 是不可变的，因此在某些情况下，`tuple` 的性能可能会比 `list` 更好，因为它不需要进行额外的内存分配和复制操作。

4. 使用场景:

- `list` 通常用于需要动态修改数据的场景，例如在循环中添加或删除元素。
- `tuple` 通常用于不需要修改的数据，例如常量或数据集。

5. 方法:

- `list` 提供了更多的方法,

如何评价复旦大学管理学院的刘沛霖？

[illegible]

刘沛霖，复旦大学管理学院教授，博士生导师，复旦大学管理学院金融学系主任，复旦大学金融研究院执行院长，复旦大学金融学博士，复旦大学经济学学士。刘沛霖教授的研究领域包括金融学、金融工程、金融风险管理、金融监管、金融创新、金融发展等。他曾在国内外知名期刊发表多篇学术论文，并主持多项国家级和省部级科研项目。刘沛霖教授还担任多个学术组织的委员和理事，如中国金融学会理事、中国金融教育发展基金会理事、中国金融教育发展基金会金融教育专业委员会副主任委员等。他还在多个学术期刊担任编委和审稿人，如《金融研究》、《经济研究》、《管理科学学报》、《管理科学学报》、《管理科学》、《管理科学与工程》、《管理科学与工程学报》、《管理科学与工程》、《管理科学与工程》、《管理科学与工程》、《管理科学与工程》。

3B模型表述更为书面和规范化

	理学等。他是一位非常有才华和创造力的学者，他的研究领域包括组织行为、领导力、管理心理学等。他是一位非常有才华和创造力的学者，他的研究领域包括组织行为、领导力、管理心理学等。他是一位非常有才华和创造力的学者，他的研究领域包括组织行为、	程》、《管理科学与工程》、《管理科学与工程》、《管理科学与工程》、《管理科学与工程》、《管理科学与工程》、《管理科学与工程》、	
--	--	---	--

7. 自检结果

代码块

```
1 python eval/run.py
```

测试项	结果	说明
lora_param_count	✓ 通过	trainable ratio = 0.11%
loss_masking	✓ 通过	mask_ratio = 55.6%
sft_vs_base	✓ 通过	已有 ckpt/sft/lora.pt ，需重跑 eval/run.py 确认

注：若 eval/result.json 中 sft_vs_base 仍为 false，是因为自检在 SFT 训练完成前运行；重跑即可。